Machine Learning for Bioinformatics

Exercise Sheet

Freie Universität Berlin

Please note that the jupyter notebook must be submitted
along with the exercise sheet![1]

Name:		Matriculation no.:	
Name:		Matriculation no.:	

Introduction to PyTorch

PyTorch is a library for numeric computing with tensors, automatic differentiation, and trainable models. In this exercise, we first use PyTorch as a tensor library on the SMS spam dataset. We then reimplement the previously introduced Naive Bayes spam classifier with tensor operations and inspect the effect of the classification threshold. Finally, we introduce the standard PyTorch model interface with `torch.nn.Module`, `BCELoss`, and an optimizer.

Assignment 1. *Introduction to PyTorch (optional)*

1. If you have never worked with PyTorch before you might want to go through the first assignment. It explains tensors, vectorized operations, automatic differentiation, and optimizers. Points: 0

Assignment 2. *SMS data as tensors* We use the SMS spam dataset from the earlier spam classification exercise and represent messages as binary word-occurrence vectors.

1. What is the number of ham and spam messages in the corpus? Points: 1

   Ham: Spam:

2. How often do the retained words *free*, *win*, and *call* occur in the binary message matrix? Points: 1

   free: win: call:

Assignment 3. *Naive Bayes with PyTorch tensors* Reimplement the Bernoulli Naive Bayes spam classifier with PyTorch tensor operations.

1. Complete the implementation of the Naive Bayes parameter estimates and prediction rule. Points: 0

2. Report the test-set accuracy of the Naive Bayes classifier. Points: 1

   Accuracy:

[1] No points are awarded if an answer is only partially correct. Answers must be supported by results from the jupyter notebook.

3. Which threshold gives the largest validation accuracy? Points: 1

   Threshold: Accuracy:

4. Specify the values of the Naive Bayes confusion matrix. Points: 1

5. Which retained word has the largest spam log-odds? Points: 1

   Word:

Assignment 4. *Trainable PyTorch model* Solve the same spam-classification task with a linear PyTorch model, a sigmoid output, `BCELoss`, and an optimizer.

1. Complete the implementation of the *DetectSpamNetwork* model and the training loop. Points: 0

2. Report the test-set accuracy of the trained model. Points: 1

   Accuracy:

3. Specify the values of the trained model confusion matrix. Points: 1

Maximum number of points: 8